

1. List ADT

1.1 . Abstract Data Type (ADT)

An Abstract Data Type (ADT) is a mathematical model for a certain class of data structures. It is defined by the operations that can be performed on it, rather than its implementation. Examples: Stack (LIFO), Queue (FIFO).

1.1 Stack ADT Example

Stack operations include: push(), pop(), peek(), isEmpty().

1.2 List ADT

A List ADT is an ordered sequence of elements. It supports position-based access (indexing) and allows duplicate elements.

1.3 Array-Based Implementation

Lists can be implemented using arrays. Key properties include:

- Contiguous memory block
- Fixed-size limitation
- Indexed element storage
- Tracking current size with a variable

1.4 Core Operations

Insertions and deletions can be done at the end or at a specific index.

Insertion at index involves shifting elements. Worst-case time: $O(N)$.

Resizing involves creating a larger array and copying elements: $O(N)$.

Removing involves shifting elements left. Time complexity: $O(N)$.

1.5 Code Snippets

Insert at index:

```
void insertAt(ArrayADT *arr, int index, int value) {
    if (index < 0 || index > arr->size || arr->size >= MAX_SIZE) {
        printf("Invalid index or array full.\n");
        return;
    }
    for (int i = arr->size; i > index; i--) {
        arr->data[i] = arr->data[i - 1];
    }
    arr->data[index] = value;
    arr->size++;
}
```

Delete at index:

```
void deleteAt(ArrayADT *arr, int index) {
    if (index < 0 || index >= arr->size) {
        printf("Invalid index.\n");
        return;
    }
    for (int i = index; i < arr->size - 1; i++) {
        arr->data[i] = arr->data[i + 1];
    }
    arr->size--;
}
```

1.6 Other Key Operations and Complexity

- get(index): $O(1)$
- set(index, value): $O(1)$
- size(): $O(1)$
- isEmpty(): $O(1)$

1.7 Advantages and Disadvantages

Advantages:

- $O(1)$ random access
- Memory locality and cache-friendly

Disadvantages:

- $O(N)$ for insert/delete in the middle
- Fixed capacity and costly resizing

1.8 Full Code Implementation

```
#include <stdio.h>
#define MAX_SIZE 100

typedef struct {
    int data[MAX_SIZE];
    int size;
} ArrayADT;
```

```

void init(ArrayADT *arr) {
    arr->size = 0;
}

void insertEnd(ArrayADT *arr, int value) {
    if (arr->size >= MAX_SIZE) {
        printf("Array is full. Cannot insert.\n");
        return;
    }
    arr->data[arr->size++] = value;
}

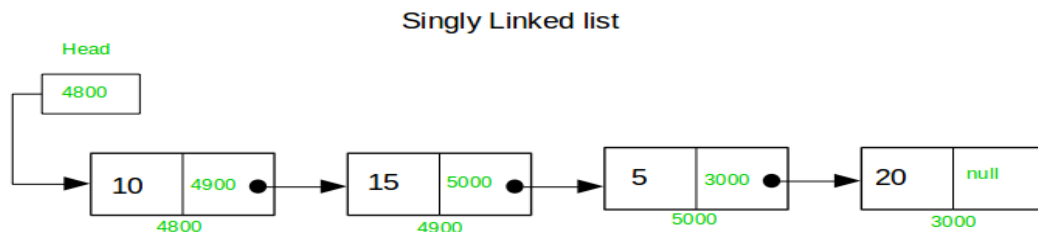
void deleteAt(ArrayADT *arr, int index) {
    if (index < 0 || index >= arr->size) {
        printf("Invalid index.\n");
        return;
    }
    for (int i = index; i < arr->size - 1; i++) {
        arr->data[i] = arr->data[i + 1];
    }
    arr->size--;
}

int search(ArrayADT *arr, int value) {
    for (int i = 0; i < arr->size; i++) {
        if (arr->data[i] == value) {
            return i; }
    }
    return -1;
}

int main() {
    ArrayADT arr;    init(&arr);
    insertEnd(&arr, 10);
    insertEnd(&arr, 20);
    insertEnd(&arr, 30);
    // display code would go here
    insertAt(&arr, 1, 15);    deleteAt(&arr, 2);
    int idx = search(&arr, 15);
    if (idx != -1)    printf("Element found at index: %d\n", idx);
    else
        printf("Element not found.\n");
    return 0;
}

```

2. A single linked list is a linear data structure where each element, or node, contains data and a pointer (or link) to the next node in the sequence. Unlike arrays, linked lists do not require contiguous memory allocation; each node can be stored in a separate memory location.



Characteristics of a Singly Linked List

- Each node holds a single value and a reference to the next node in the list.
- The list has a head or start, which is a reference to the first node in the list. We can access all items of a list using the start or head node.
- The nodes are not stored in a contiguous block of memory, but instead, each node holds the address of the next node in the list.
- Accessing elements in a singly linked list requires traversing the list from the head to the desired node, as there is no direct access to a specific node in memory.

Operations on Linked Lists

Insertion: Adding a new node to a linked list involves adjusting the pointers of the existing nodes to maintain the proper sequence. Insertion can be performed at the beginning, end, or any position within the list

i. Insertion at the beginning of Single Linked List

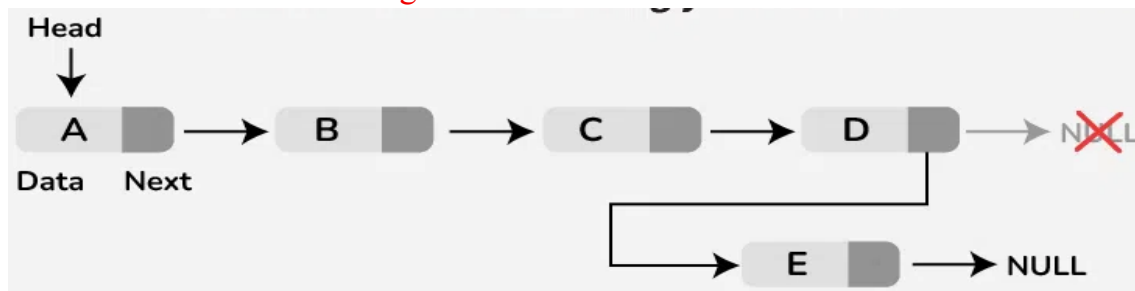


Algorithm:

- Make the first node of Linked List linked to the new node
- Remove the head from the original first node of Linked List
- Make the new node as the Head of the Linked List.

Complexity - $O(1)$ Constant-time pointer updates.

ii. Insertion at the end of Single Linked List

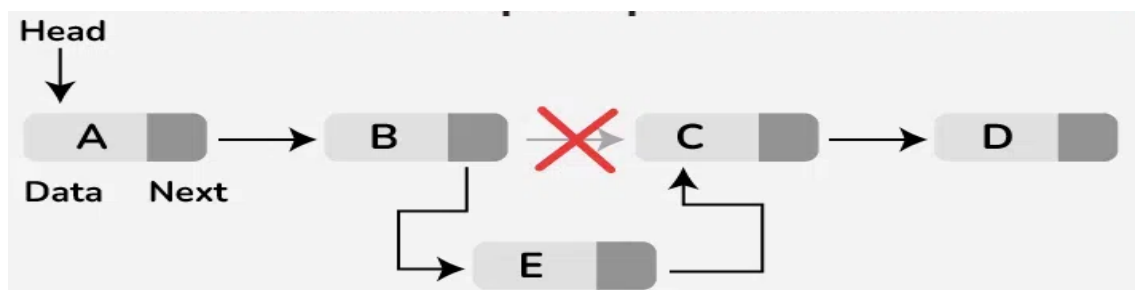


Algorithm:

- Create a new node and set its next pointer as NULL since it will be the last node.
- Store the head reference in a temporary variable
- If the Linked List is empty, make the new node as the head and return
- Else traverse till the last node
- Change the next pointer of the last node to point to the new node

Complexity : $O(n)$ Traversal required to find the last node.

iii. Insert a node at a specific position in a linked list



Algorithm

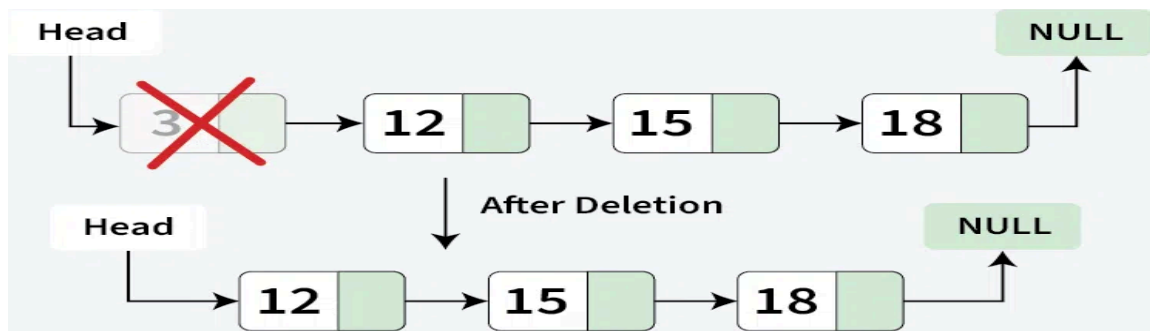
- Initialize a variable , say **curr** points to **head** and allocate the memory to the **new node** with the given **data**.
- Traverse the Linked list using **curr** pointer upto **position-1** nodes.

- If **curr's** next is not **null** , then next pointer of the **new node** points to the **next** of **curr** node.
- The **next** pointer of **current node** points to the **new node**.
- return the **head** of linked list.

Complexity - $O(n)$ Traversal to the desired position

Deletion: Removing a node from a linked list requires adjusting the pointers of the neighboring nodes to bridge the gap left by the deleted node. Deletion can be performed at the beginning, end, or any position within the list.

i. Deletion at beginning (Removal of first node) in a Linked List



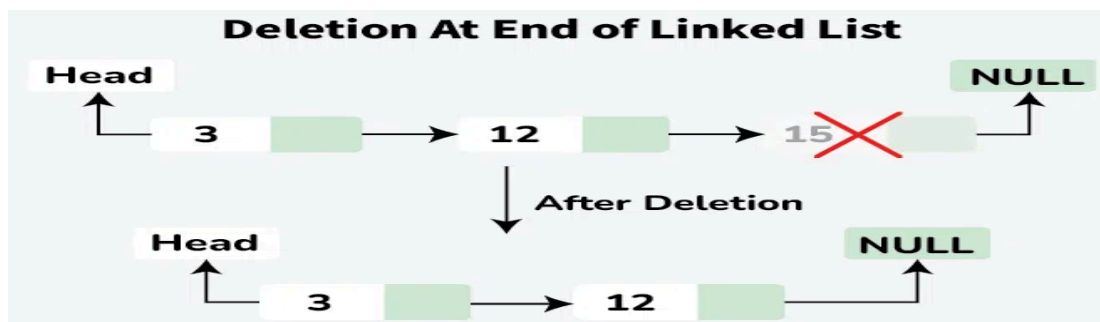
Algorithm

To remove the first node of a *linked list*,

- store the current **head** in a temporary variable (**temp**),
- move the **head** pointer to the next node,
- **delete** the temporary **head** node and
- finally , return the new **head** of the linked list.

Complexity - $O(1)$, Constant-time pointer update

ii. Deletion at End of Linked List



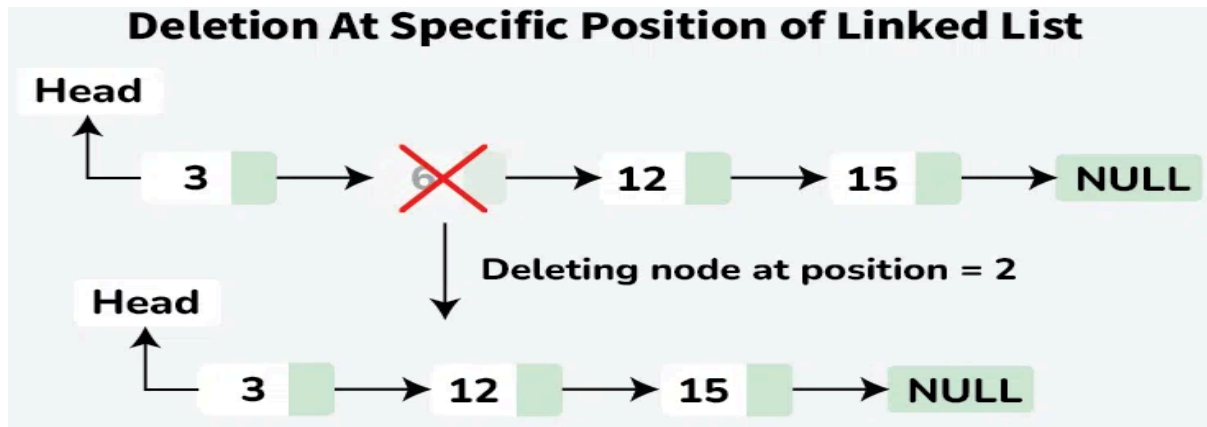
To perform the deletion operation at the end of linked list,

- we need to traverse the list to find the **second last node**,
- then set its next pointer to **null**.

- If the list is empty then there is no node to delete or has only one node then point **head** to **null**.

Complexity - $O(n)$, Traversal required to find the second last node.

iii. Delete a Linked List node at a given position



To perform the deletion,

- If the position is 1, we update the **head** to point to the **next node** and delete the current head.
- For other positions, we traverse the list to reach the node just before the specified **position**.
- If the target node exists, we adjust the next of this previous node to point to next of **next nodes**, which will result in skipping the target node.

Complexity - $O(n)$, Traversal to the desired position

Searching: Searching for a specific value in a linked list involves traversing the list from the head node until the value is found or the end of the list is reached.

i. Search an element in a Linked List

Algorithm

- Initialize a node pointer, **curr = head**.
- Do following while current is not NULL
 - o If the current value (i.e., **curr->key**) is equal to the key being searched return true.
 - o Otherwise, move to the next node (**curr = curr->next**).
- If the key is not found, return false

Complexity - $O(n)$, Traversal through the list to find the desired value

Common Errors:

The most common errors encountered are "Memory access violation" or "Segmentation fault". These messages usually indicate that a pointer variable contains a bogus or invalid address, often because the variable was not properly initialized. Another common mistake is misunderstanding when to use `malloc` to allocate memory for a new cell or node. Declaring a pointer to a structure does not create the structure itself; it only allocates enough space to store the address where the structure might be located.

When memory is no longer needed, we can use the `free` command to reclaim the space. However, after calling `free`, the pointer `p` still holds the same address, but the data at that address becomes undefined, and using it may lead to errors. Also, note that `malloc` may return `NULL` if the system is unable to satisfy the memory request for a new node. In such cases, it's important to always check if the allocation was successful before using the pointer.

Program on all the operations of Single Linked List

```
#include<stdio.h>
#include<malloc.h>
struct node
{
int data;
struct node *link;
};
struct node *start;
void create (int n)
{
    start=(struct node*)malloc(sizeof(struct node));
    printf("Enter the Data::");
    scanf("%d",&(start->data));
    start->link=NULL;
    struct node *temp, *t;
    t=start;
    for(int i=2;i<=n;i++)
    {
        temp=(struct node*)malloc(sizeof(struct node));
```

```

        printf("Enter the Data::");
        scanf("%d",&(temp->data));
        temp->link=NULL;
        t->link=temp;
        t=t->link;
    }
}

void display( )
{
    struct node *temp;
    temp=start;
    while(temp!=NULL)
    {
        printf("%3d",temp->data);
        temp=temp->link;
    }
}

void insert_end( )
{
    struct node *t, *temp;
    t=start;
    temp=(struct node*)malloc(sizeof(struct node));
    printf("\nEnter the Data at the end::");
    scanf("%d",&(temp->data));
    temp->link=NULL;
    while(t->link!=NULL)
    {
        t=t->link;
    }
    t->link=temp;
}

void insert_beg( )
{
    struct node *temp;
    temp=(struct node*)malloc(sizeof(struct node));
    printf("\nEnter the Data at the beginning::");
    scanf("%d",&(temp->data));
    temp->link=NULL;
    temp->link=start;
    start=temp;
}

void insert_pos(int pos)
{
    struct node *temp, *t;

```

```

temp=(struct node*)malloc(sizeof(struct node));
printf("\nEnter the Data at the middle position:");
scanf("%d",&(temp->data));
temp->link=NULL;
t=start;
int p=1;
while(p<pos-1)
{
    p++;
    t=t->link;
}
temp->link=t->link;
t->link=temp;
}

void del_beg( )
{
    struct node *t;
    t=start;
    start=t->link;
    printf("\nThe deleted element is %d\n",t->data);
    free(t);
}

void del_end( )
{
    struct node *temp1, *temp2;
    temp1=start;
    temp2=temp1->link;
    while(temp2->link!=NULL)
    {
        temp1=temp1->link;
        temp2=temp2->link;
    }
    temp1->link=NULL;
    printf("\nThe deleted element is %d\n", temp2->data);
    free(temp2);
}

del_pos(int pos)
{
    struct node *temp1, *temp2;
    temp1=start;
    temp2=start->link;
    int p=1;

```

```

        while(p<pos-1)
        {
            p++;
            temp1=temp1->link;
            temp2=temp2->link;
        }
        temp1->link=temp2->link;
        printf("\nThe deleted element is %d\n",temp2->data);
        free(temp2);
    }

    int main( )
    {
        int po,n=10;
        create(n);
        display();
        insert_end();
        display();
        insert_beg();
        display();
        printf("\n Enter the position, where to insert the element in the linked list::");
        scanf("%d",&po);
        insert_pos(po);
        display();
        del_beg();
        display();
        del_end();
        display();
        del_pos(po);
        display();
    }

```

Using Switch case(Insertion in Linked List)

```

#include<stdio.h>
#include<malloc.h>
struct node
{
    int info;
    struct node *next;
};

```

```

struct node *start=NULL;
void creat();
void display();
void insert_beg();
void insert_end();
void insert_pos();
int solve (struct node *);
int main()
{
    int en, ch, p;
    printf("choice 1 is for create\n");
    printf("choice 2 is for insert at the beginning\n");
    printf("choice 3 is for insert at the end\n");
    printf("choice 4 is for insert at any position\n");
    printf("choice 5 is for display\n");
    do
    {
        printf("Enter the choice:");
        scanf("%d",&ch);
        switch(ch)
        {
            case 1:creat();
                    break;
            case 2:insert_beg();
                    break;
            case 3:insert_end();
                    break;
            case 4:insert_pos();
                    break;
            case 5:display();
                    break;
            default:printf("Exit\n");
        }
        printf("Enter 1 to continue:");
        scanf("%d",&en);
    }while(en==1);
    p=solve(start);
    printf("%d",p);
}

```

```

void create()

```

```

{
    int data;

```

```

start=(struct node *)malloc(sizeof(struct node));
printf("\nEnter the first element");
scanf("%d",&data);
start->info=data;
//start->next=NULL;
struct node *temp, *t=start;
for(int i=2;i<6;i++)
{
    temp=(struct node *)malloc(sizeof(struct node));
    printf("\n Enter the element to be inserted from 2nd element onwards");
    scanf("%d",&data);
    temp->info=data;
    temp->next=NULL;
    t->next=temp;
    t=t->next;
}
}

```

void display()

```

{
struct node *p=start;
printf("\n\n*****LINK LIST*****\n\n");
while(p!=NULL)
{
    printf("%d\n", p->info);
    p=p->next;
}
}

```

void insert_beg()

```

{
    int data;
    struct node *temp;
    temp=(struct node *)malloc(sizeof(struct node));
    printf("\n Enter the element to be inserted at the beggning ");
    scanf("%d",&data);
    temp->info=data;
    temp->next=NULL;
    temp->next=start;
    start=temp;
}

```

void insert_end()

```

{
    int data;

```

```

        struct node *temp, *t;
        temp=(struct node *)malloc(sizeof(struct node));
        printf("\n Enter the element to be inserted at the end ");
        scanf("%d",&data);
temp->info=data;
temp->next=NULL;
t=start;
while(t->next!=NULL)
    {
        t=t->next;
    }
t->next=temp;
}

```

void insert_pos()

```

{
int pos,data;
struct node *temp, *t;
printf("Enter the position of the element");
scanf("%d",&pos);
temp=(struct node *)malloc(sizeof(struct node));
printf("\n Enter the element to be inserted at the end ");
scanf("%d",&data);
temp->info=data;
temp->next=NULL;
t=start;
int p=1;
while(p<pos-1)
    {
        p++;
        t=t->next;
    }
temp->next=t->next;
t->next=temp;
}

```

Using Switch case(Deletion in Linked List)

```

#include<stdio.h>
#include<malloc.h>
    struct node
    {

```

```

        int info;
        struct node *next;
    };
struct node *start=NULL;
void create();
void display();
void delete_beg();
void delete_end();
void delete_pos();
int main()
{
    int en, ch;
    printf("choice 1 is for create\n");
    printf("choice 2 is for delete at the beginning\n");
    printf("choice 3 is for delete at the end\n");
    printf("choice 4 is for delete at the any position\n");
    printf("choice 5 is for display\n");
    do
    {
        printf("\nEnter the choice:");
        scanf("%d",&ch);
        switch(ch)
        {
            case 1:creat();
                    break;
            case 2:delete_beg();
                    break;
            case 3:delete_end();
                    break;
            case 4:delete_pos();
                    break;
            case 5:display();
                    break;
            default:printf("Exit\n");
        }
        printf("Enter 1 to continue:");
        scanf("%d",&en);
    }while(en==1);

```

```
        printf("test");
    }
```

void create()

```
{
    int data;
    start=(struct node *)malloc(sizeof(struct node));
    printf("\nEnter the first element");
    scanf("%d",&data);
    start->info=data;
    //start->next=NULL;
    struct node *temp, *t=start;
    for(int i=2;i<6;i++)
    {
        temp=(struct node *)malloc(sizeof(struct node));
        printf("\n Enter the element to be inserted from 2nd element onwards");
        scanf("%d",&data);
        temp->info=data;
        temp->next=NULL;
        t->next=temp;
        t=t->next;
    }
}
```

void display()

```
{
    struct node *p=start;
    printf("\n\n*****LINK LIST*****\n\n");
    while(p!=NULL)
    {
        printf("%d\n", p->info);
        p=p->next;
    }
}
```

void delete_beg()

```
{
    struct node *temp;
    temp=start;
```

```

        start=temp->next;
        printf("The element deleted is %d\n",temp->info);
        free(temp);
    }
    void delete_end()
    {
        struct node *temp1, *temp2;
        temp1=start;
        temp2=start->next;
        while(temp2->next!=NULL)
        {
            temp1=temp1->next;
            temp2=temp2->next;
        }
        temp1->next=NULL;
        printf("The element deleted is %d\n",temp2->info);
        free(temp2);
    }

```

```

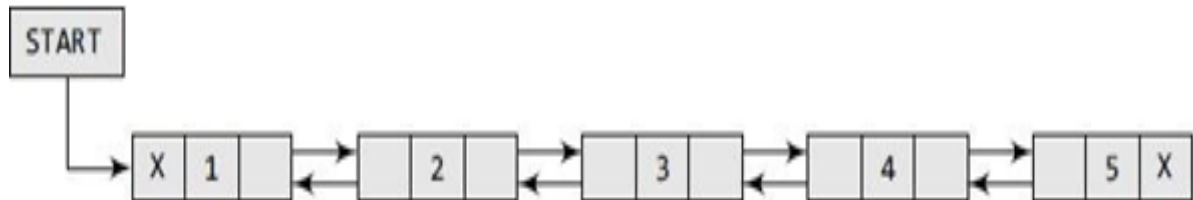
    void delete_pos()
    {
        int pos;
        struct node *temp1, *temp2;
        temp1=start;
        temp2=start->next;
        printf("Enter the position of the element");
        scanf("%d",&pos);
        int p=1;
        while(p<pos-1)
        {
            p++;
            temp1=temp1->next;
            temp2=temp2->next;
        }
        temp1->next=temp2->next;
        printf("The element deleted is %d\n",temp2->info);
        free(temp2);
    }

```

1.3 DOUBLY LINKED LISTS

Definition

A doubly linked list or a two-way linked list is a more complex type of linked list which contains a pointer to the next as well as the previous node in the sequence. Therefore, it consists of three parts—data, a pointer to the next node, and a pointer to the previous node as shown in Figure



The structure of a doubly linked list can be given as,

struct node

```
{
    struct node *prev;
    int data;
    struct node *next;
};
```

The PREV field of the first node and the NEXT field of the last node will contain NULL. The PREV field is used to store the address of the preceding node, which enables us to traverse the list in the backward direction. Doubly linked list calls for more space per node and more expensive basic operations. However, a doubly linked list provides the ease to manipulate the elements of the list as it maintains pointers to nodes in both the directions (forward and backward). The main advantage of using a doubly linked list is that it makes searching twice as efficient.

Memory representation of a doubly linked list

START				
1				
	1	H	-1	3
	2			
	3	E	1	6
	4			
	5			
	6	L	3	7
	7	L	6	9
	8			
	9	O	7	-1

Variable START is used to store the address of the first node. In this example, START = 1, so the first data is stored at address 1, which is H. Since this is the first node, it has no previous node and hence stores NULL or -1 in the PREV field. We will traverse the list until we reach a position where the NEXT entry contains -1 or NULL. This denotes the end of the linked list. When we traverse the DATA and NEXT in this manner, we will finally see that the linked list in the above example stores characters that when put together form the word HELLO.

Inserting a New Node in a Doubly Linked List

Case 1: The new node is inserted at the beginning.

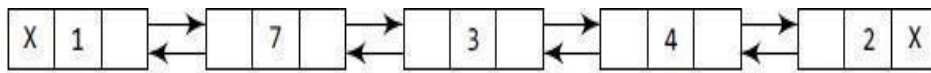
Case 2: The new node is inserted at the end.

Case 3: The new node is inserted after a given node.

Case 4: The new node is inserted before a given node.

Case 1: The new node is inserted at the beginning:

We want to add a new node with data 9 as the first node of the list. Then the following changes will be done in the linked list.

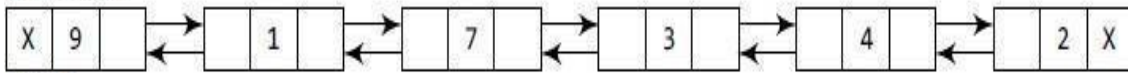


START

Allocate memory for the new node and initialize its DATA part to 9 and PREV field to NULL.



Add the new node before the START node. Now the new node becomes the first node of the list.



START

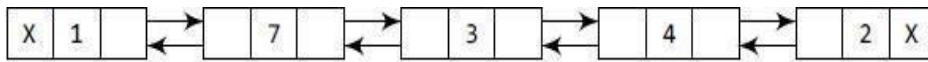
Algorithm to insert a new node at the beginning

```
Step 1: IF AVAIL = NULL
        Write OVERFLOW
        Go to Step 9
    [END OF IF]
Step 2: SET NEW_NODE = AVAIL
Step 3: SET AVAIL = AVAIL -> NEXT
Step 4: SET NEW_NODE -> DATA = VAL
Step 5: SET NEW_NODE -> PREV = NULL
Step 6: SET NEW_NODE -> NEXT = START
Step 7: SET START -> PREV = NEW_NODE
Step 8: SET START = NEW_NODE
Step 9: EXIT
```

- In Step 1, we first check whether memory is available for the new node.
- If the free memory has exhausted, then an OVERFLOW message is printed.
- Otherwise, if free memory cell is available, then we allocate space for the new node.
- Set its DATA part with the given VAL and the NEXT part is initialized with the address of the first node of the list, which is stored in START.
- Now, since the new node is added as the first node of the list, it will now be known as the START node, that is, the START pointer variable will now hold the address of NEW_NODE.

Case 2: The new node is inserted at the end.

We want to add a new node with data 9 as the last node of the list. Then the following changes will be done in the linked list.

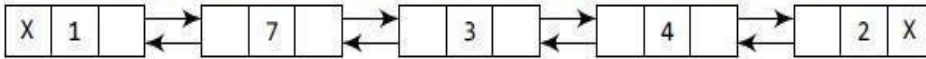


START

Allocate memory for the new node and initialize its DATA part to 9 and its NEXT field to NULL.

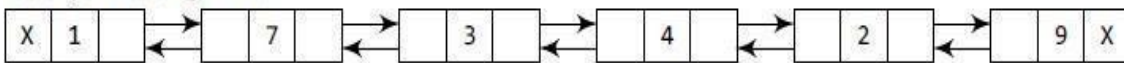


Take a pointer variable PTR and make it point to the first node of the list.



START, PTR

Move PTR so that it points to the last node of the list. Add the new node after the node pointed by PTR.



START

PTR

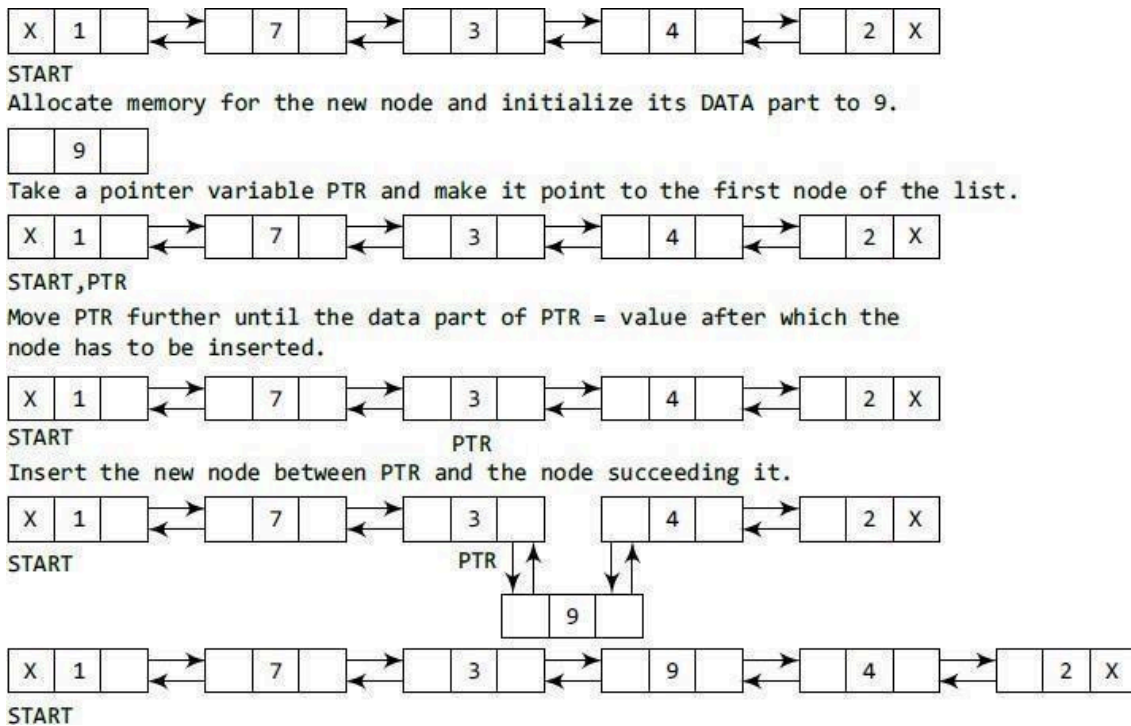
Algorithm to insert a new node at the end

```
Step 1: IF AVAIL = NULL
        Write OVERFLOW
        Go to Step 11
    [END OF IF]
Step 2: SET NEW_NODE = AVAIL
Step 3: SET AVAIL = AVAIL -> NEXT
Step 4: SET NEW_NODE -> DATA = VAL
Step 5: SET NEW_NODE -> NEXT = NULL
Step 6: SET PTR = START
Step 7: Repeat Step 8 while PTR -> NEXT != NULL
Step 8:     SET PTR = PTR -> NEXT
    [END OF LOOP]
Step 9: SET PTR -> NEXT = NEW_NODE
Step 10: SET NEW_NODE -> PREV = PTR
Step 11: EXIT
```

- In Step 6, we take a pointer variable PTR and initialize it with START.
- In the while loop, we traverse through the linked list to reach the last node.
- Once we reach the last node, in Step 9, we change the NEXT pointer of the last node to store the address of the new node.
- Remember that the NEXT field of the new node contains NULL which signifies the end of the linked list.
- The PREV field of the NEW_NODE will be set so that it points to the node pointed by PTR (now the second last node of the list).

Case 3: The new node is inserted after a given node.

We want to add a new node with value 9 after the node containing 3.



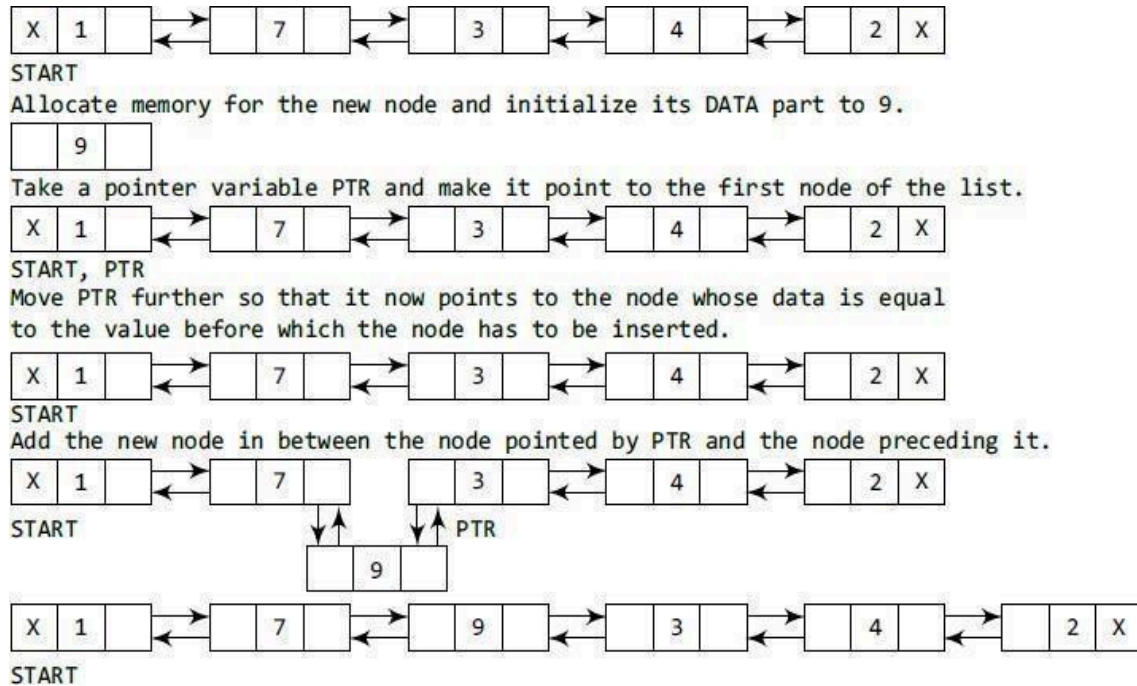
Algorithm to insert a new node after a given node

```

Step 1: IF AVAIL = NULL
        Write OVERFLOW
        Go to Step 12
    [END OF IF]
Step 2: SET NEW_NODE = AVAIL
Step 3: SET AVAIL = AVAIL → NEXT
Step 4: SET NEW_NODE → DATA = VAL
Step 5: SET PTR = START
Step 6: Repeat Step 7 while PTR → DATA != NUM
Step 7:     SET PTR = PTR → NEXT
    [END OF LOOP]
Step 8: SET NEW_NODE → NEXT = PTR → NEXT
Step 9: SET NEW_NODE → PREV = PTR
Step 10: SET PTR → NEXT = NEW_NODE
Step 11: SET PTR → NEXT → PREV = NEW_NODE
Step 12: EXIT
    
```

- In Step 5, we take a pointer PTR and initialize it with START.
- That is, PTR now points to the first node of the linked list. In the while loop, we traverse through the linked list to reach the node that has its value equal to NUM.
- We need to reach this node because the new node will be inserted after this node.
- Once we reach this node, we change the NEXT and PREV fields in such a way that the new node is inserted after the desired node.
- Case 4: The new node is inserted before a given node.

We want to add a new node with value 9 before the node containing 3.



Algorithm to insert a new node before a given node

```

Step 1: IF AVAIL = NULL
        Write OVERFLOW
        Go to Step 12
    [END OF IF]
Step 2: SET NEW_NODE = AVAIL
Step 3: SET AVAIL = AVAIL->NEXT
Step 4: SET NEW_NODE->DATA = VAL
Step 5: SET PTR = START
Step 6: Repeat Step 7 while PTR->DATA != NUM
Step 7:     SET PTR = PTR->NEXT
    [END OF LOOP]
Step 8: SET NEW_NODE->NEXT = PTR
Step 9: SET NEW_NODE->PREV = PTR->PREV
Step 10: SET PTR->PREV = NEW_NODE
Step 11: SET PTR->PREV->NEXT = NEW_NODE
Step 12: EXIT

```

- In Step 1, we first check whether memory is available for the new node.
- In Step 5, we take a pointer variable PTR and initialize it with START.
- That is, PTR now points to the first node of the linked list.
- In the while loop, we traverse through the linked list to reach the node that has its value equal to NUM.
- We need to reach this node because the new node will be inserted before this node.
- Once we reach this node, we change the NEXT and PREV fields in such a way that the new node is inserted before the desired node.

Deleting a Node from a Doubly Linked List

In this section, we will see how a node is deleted from an already existing doublylinked list

Case 1: The first node is deleted.

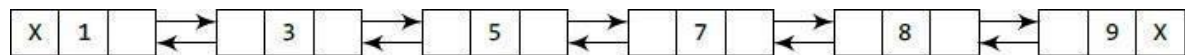
Case 2: The last node is deleted.

Case 3: The node after a given node is deleted.

Case 4: The node before a given node is deleted.

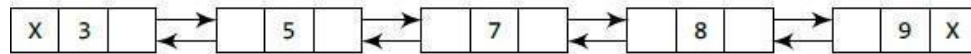
Case 1: The first node is deleted.

When we want to delete a node from the beginning of the list, then the following changes will be done in the linked list



START

Free the memory occupied by the first node of the list and make the second node of the list as the START node.



START

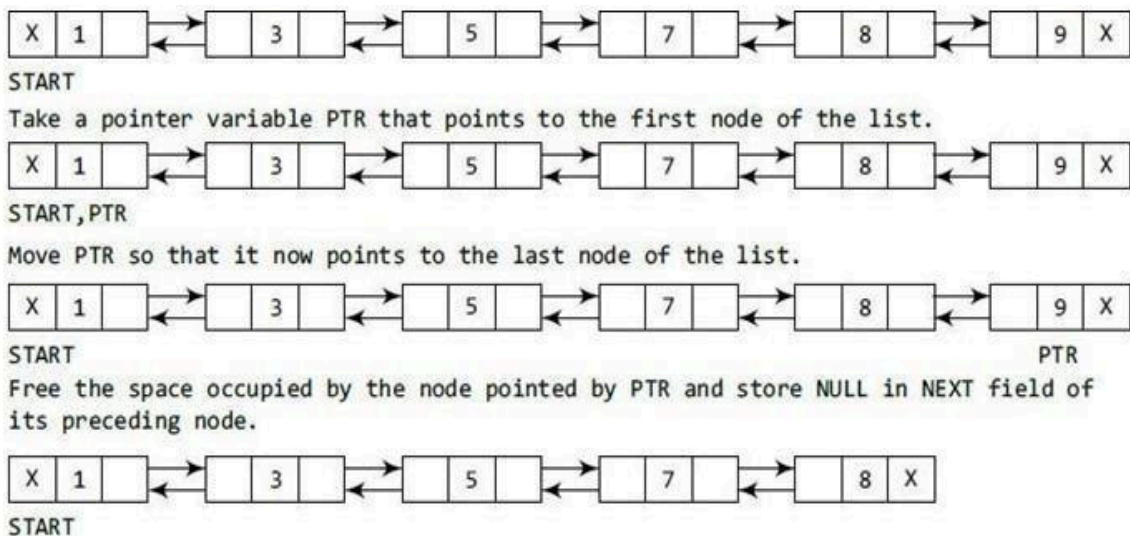
Algorithm to delete the first node

```
Step 1: IF START = NULL
        Write UNDERFLOW
        Go to Step 6
    [END OF IF]
Step 2: SET PTR = START
Step 3: SET START = START -> NEXT
Step 4: SET START -> PREV = NULL
Step 5: FREE PTR
Step 6: EXIT
```

- In Step 1 of the algorithm, we check if the linked list exists or not.
- If $START = NULL$, then it signifies that there are no nodes in the list and the control is transferred to the last statement of the algorithm.
- However, if there are nodes in the linked list, then we use a temporary pointer variable PTR that is set to point to the first node of the list. For this, we initialize PTR with START that stores the address of the first node of the list.
- In Step 3, START is made to point to the next node in sequence and finally the memory occupied by PTR (initially the first node of the list) is freed and returned to the free pool.

Case 2: The last node is deleted.

We want to delete the last node from the linked list, then the following changes will be done in the linked list.



Algorithm to delete the last node

```

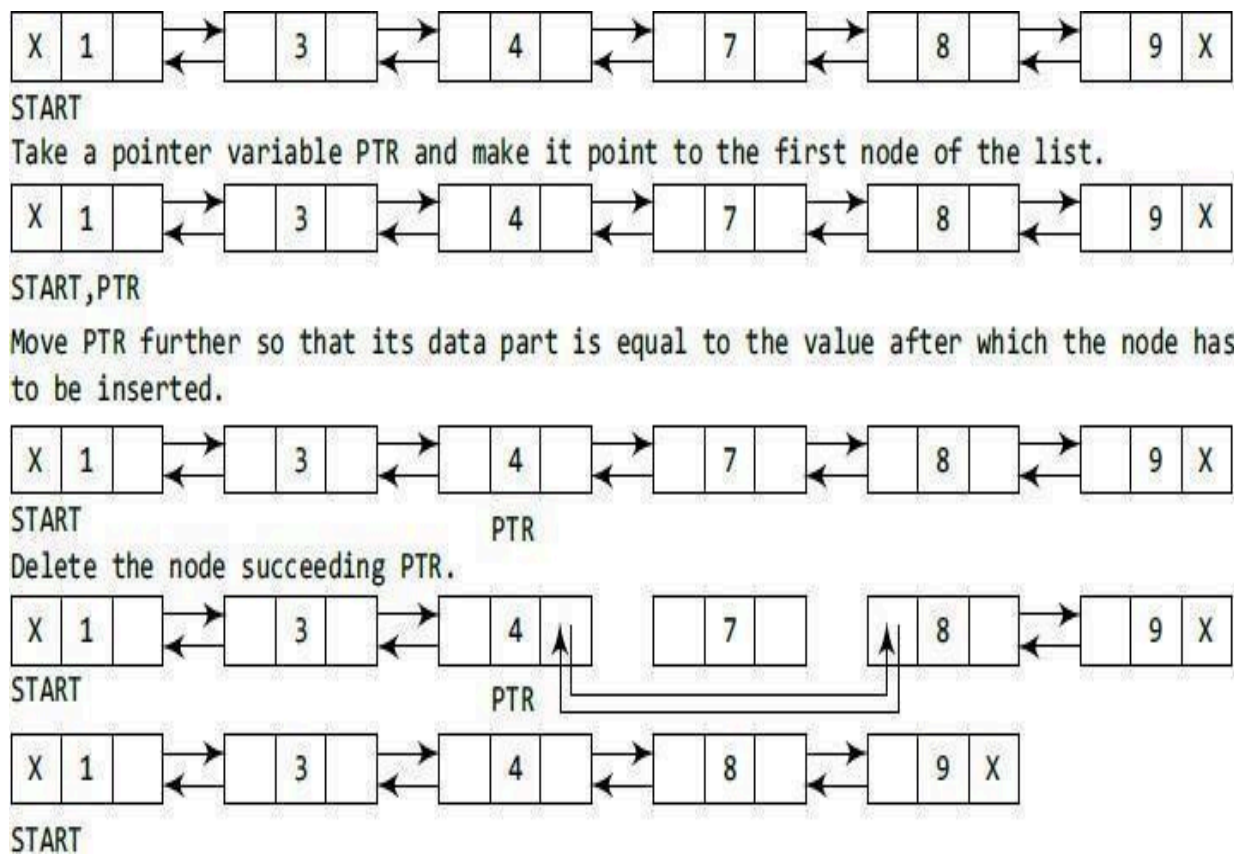
Step 1: IF START = NULL
        Write UNDERFLOW
        Go to Step 7
    [END OF IF]
Step 2: SET PTR = START
Step 3: Repeat Step 4 while PTR->NEXT != NULL
Step 4:     SET PTR = PTR->NEXT
    [END OF LOOP]
Step 5: SET PTR->PREV->NEXT = NULL
Step 6: FREE PTR
Step 7: EXIT

```

- In Step 2, we take a pointer variable PTR and initialize it with START.
- That is, PTR now points to the first node of the linked list. The while loop traverses through the list to reach the last node.
- Once we reach the last node, we can also access the second last node by taking its address from the PREV field of the last node.
- To delete the last node, we simply have to set the next field of second last node to NULL, so that it now becomes the (new) last node of the linked list. The memory of the previous last node is freed and returned to the free pool.

Case 3: The node after a given node is deleted.

We want to delete the node that succeeds the node which contains data value 4. Then the following changes will be done in the linked list.



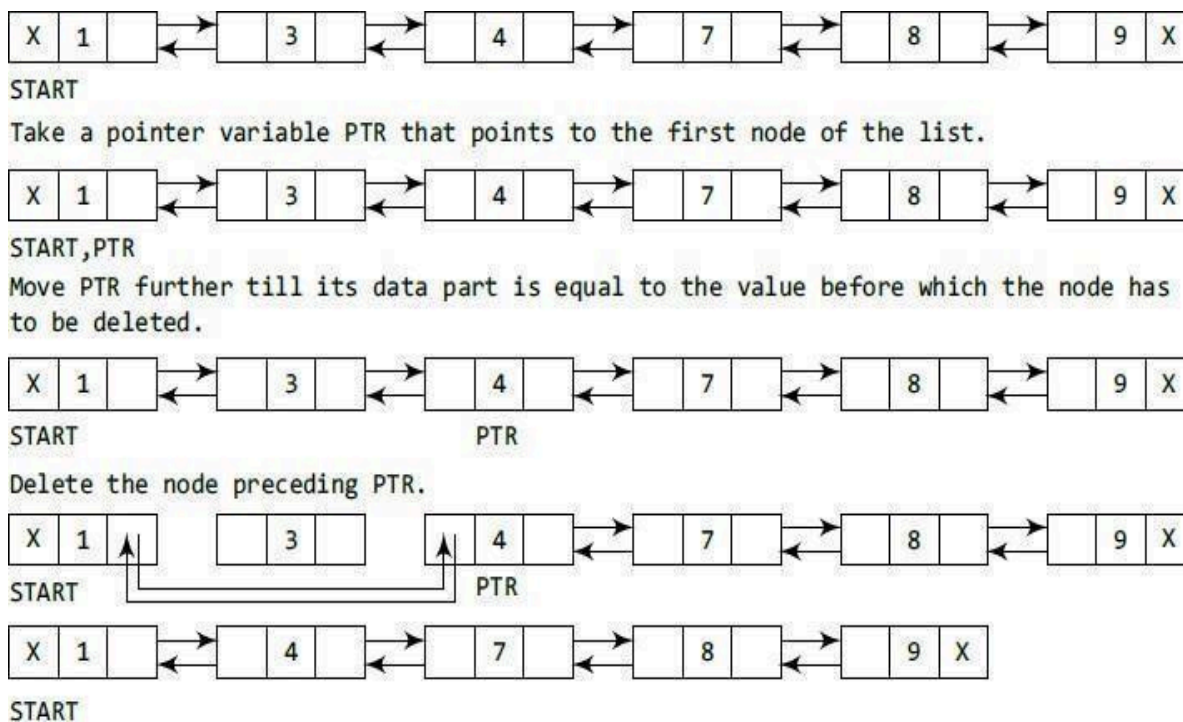
Algorithm to delete a node after a given node

```
Step 1: IF START = NULL
        Write UNDERFLOW
        Go to Step 9
    [END OF IF]
Step 2: SET PTR = START
Step 3: Repeat Step 4 while PTR->DATA != NUM
Step 4:     SET PTR = PTR->NEXT
    [END OF LOOP]
Step 5: SET TEMP = PTR->NEXT
Step 6: SET PTR->NEXT = TEMP->NEXT
Step 7: SET TEMP->NEXT->PREV = PTR
Step 8: FREE TEMP
Step 9: EXIT
```

In Step 2, we take a pointer variable PTR and initialize it with START. That is, PTR now points to the first node of the doubly linked list. The while loop traverses through the linked list to reach the given node. Once we reach the node containing VAL, the node succeeding it can be easily accessed by using the address stored in its NEXT field. The NEXT field of the given node is set to contain the contents in the NEXT field of the succeeding node. Finally, the memory of the node succeeding the given node is freed and returned to the free pool.

Case 4: The node before a given node is deleted.

Suppose we want to delete the node preceding the node with value 4



Algorithm to delete a node before a given node

```
Step 1: IF START = NULL
        Write UNDERFLOW
        Go to Step 9
    [END OF IF]
Step 2: SET PTR = START
Step 3: Repeat Step 4 while PTR->DATA != NUM
Step 4:     SET PTR = PTR->NEXT
    [END OF LOOP]
Step 5: SET TEMP = PTR->PREV
Step 6: SET TEMP->PREV->NEXT = PTR
Step 7: SET PTR->PREV = TEMP->PREV
Step 8: FREE TEMP
Step 9: EXIT
```

- In Step 2, we take a pointer variable PTR and initialize it with START.
- That is, PTR now points to the first node of the linked list.
- The while loop traverses through the linked list to reach the desired node.
- Once we reach the node containing VAL, the PREV field of PTR is set to contain the address of the node preceding the node which comes before PTR.
- The memory of the node preceding PTR is freed and returned to the free pool.

Circular Linked List - Notes

1. Introduction

A Circular Linked List is a variation of a linked list in which the last node points back to the first node. Unlike a linear linked list that terminates with a NULL pointer, a circular linked list forms a loop, making it useful in applications requiring continuous iteration, such as round-robin scheduling or buffer management. There are two types of circular linked lists:

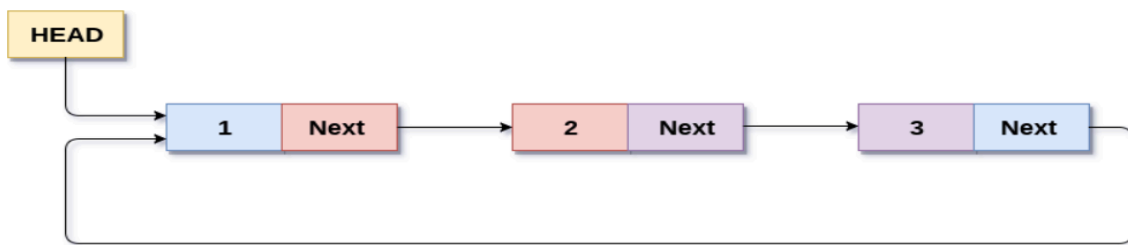
1. Circular Singly Linked List (CSLL): Each node points to the next node, and the last node points to the first.
2. Circular Doubly Linked List (CDLL): Each node points both to its next and previous nodes, with the first and last nodes connected to each other.

In this document we focused on the Circular Singly Linked List.

2. Diagram of Circular Singly Linked List

Below is a schematic representation of a Circular Singly Linked List with four nodes storing marks:

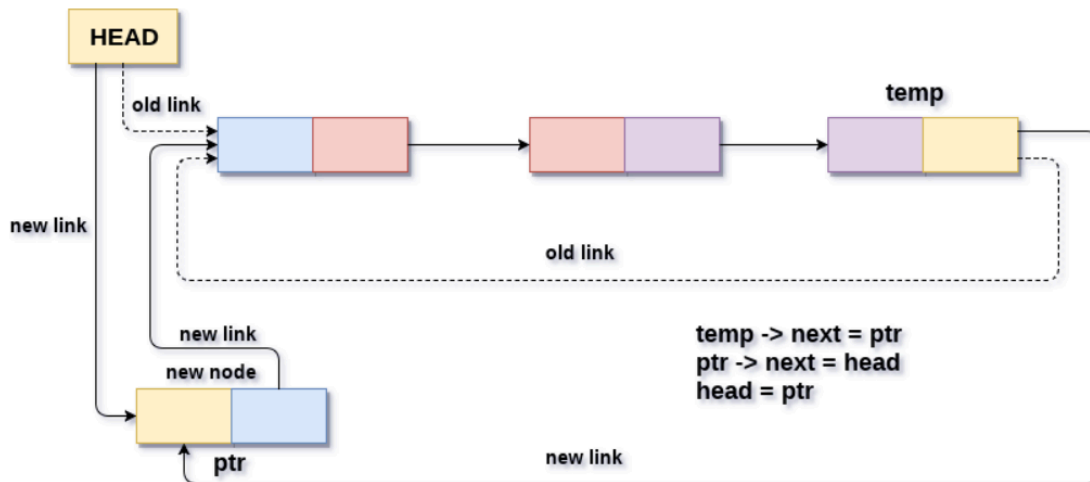
[Node1 -> Node2 -> Node3 -> Node1]



Insertion at Beginning

In a Circular Singly Linked List, inserting a node at the beginning involves creating a new node and making it the new head of the list. If the list is empty, the new node points to itself and becomes the head. If the list is non-empty, the algorithm traverses the list to find the last node, sets its next to the new node, and makes the new node's next point to the current head. Finally, the head is updated to the new node, maintaining the circular nature.

Diagram: [Insert Node → Head → Node2 → Node3 → ... → LastNode → NewNode]



C Code:

```
void beginsert() {
    struct node *ptr, *temp;
    int item;
    ptr = (struct node *)malloc(sizeof(struct node));
    if (ptr == NULL) {
        printf("\nOVERFLOW");
    } else {
        printf("\nEnter the node data? ");
        scanf("%d", &item);
        ptr->data = item;
        if (head == NULL) {
            head = ptr;
            ptr->next = head;
        } else {
            temp = head;
            while (temp->next != head)
                temp = temp->next;
            ptr->next = head;
            temp->next = ptr;
            head = ptr;
        }
        printf("\nNode inserted at beginning\n");
    }
}
```

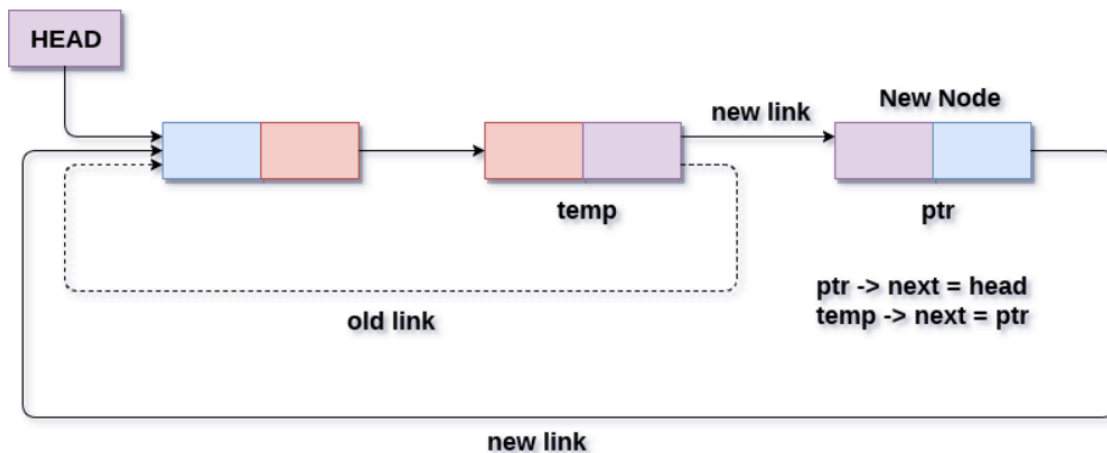
Time Complexity: $O(n)$

Explanation: In the worst case, the entire list must be traversed to locate the last node so its next pointer can be updated. This traversal takes $O(n)$ time, where n is the number of nodes in the list.

Insertion at End

Inserting a node at the end of a Circular Singly Linked List involves creating a new node and appending it after the last node. If the list is empty, the new node points to itself and becomes the head. Otherwise, the list is traversed to find the last node (whose next points to the head). The new node is linked after the last node and its next is set to the head, maintaining the circular nature.

[Head → Node2 → ... → LastNode → NewNode → Head]



C Code:

```

void endinsert() {
    struct node *ptr, *temp;
    int item;
    ptr = (struct node *)malloc(sizeof(struct node));
    if (ptr == NULL) {
        printf("\nOVERFLOW");
    } else {
        printf("\nEnter the node data? ");
        scanf("%d", &item);
        ptr->data = item;
        if (head == NULL) {
            head = ptr;
            ptr->next = head;
        } else {
            temp = head;
            while (temp->next != head)

```

```

        temp = temp->next;
        temp->next = ptr;
        ptr->next = head;
    }
    printf("\nNode inserted at end\n");
}
}

```

Time Complexity: $O(n)$

Explanation: The operation requires traversal to the last node in the list, which takes $O(n)$ time. Even though the insertion itself is constant time, the traversal dominates the complexity.

Insertion at Specific Position

To insert a node at a specific position in a Circular Singly Linked List, we create a new node and traverse the list up to the (position - 1)th node. The new node is then linked between the (position - 1)th and position-th nodes. This preserves the circular structure of the list.

[... Node(i-1) $\rightarrow$ NewNode $\rightarrow$ Node(i) ... (NewNode->next = Node(i))]

C Code:

```

void insert_position() {
    struct node *ptr, *temp;
    int item, pos, count = 1;
    ptr = (struct node *)malloc(sizeof(struct node));
    if (ptr == NULL) {
        printf("\nOVERFLOW");
        return;
    }
    printf("Enter node data and position: ");
    scanf("%d%d", &item, &pos);
    ptr->data = item;
    temp = head;
    while (count < pos - 1 && temp->next != head) {
        temp = temp->next;
        count++;
    }
    if (count != pos - 1) {
        printf("Invalid position");
        return;
    }
    ptr->next = temp->next;
    temp->next = ptr;
}

```

```

    printf("Node inserted at position");
}

```

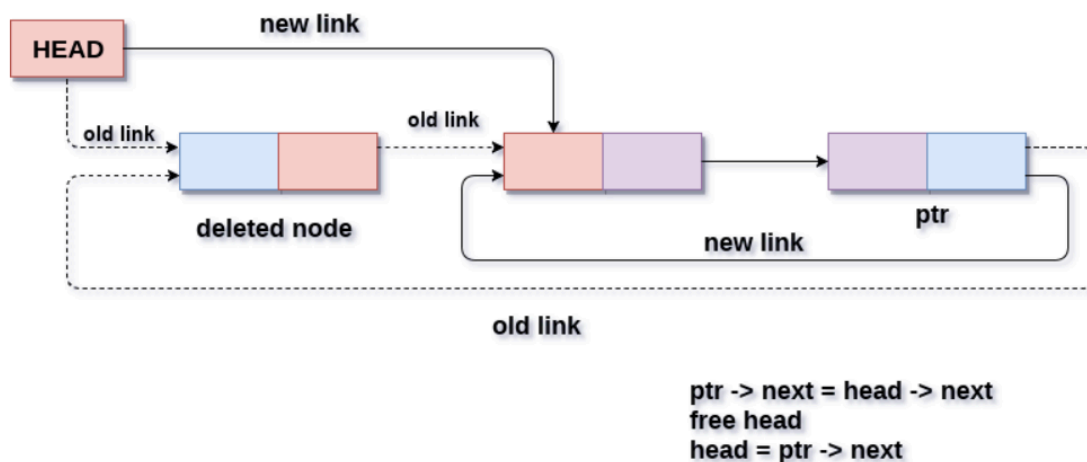
Time Complexity: $O(n)$

Explanation: The traversal to the specified position takes $O(n)$ time in the worst case (when inserting at the end). The pointer adjustments are constant time, but traversal dictates the overall complexity.

Deletion at Beginning

Deleting a node from the beginning of a Circular Singly Linked List involves removing the head node. If the list is empty, an underflow condition is reported. If there is only one node, it is freed and head is set to NULL. Otherwise, the last node is located and its next pointer is updated to point to the second node, which becomes the new head. This maintains the circular link.

Diagram: [Head(Node1) to be deleted → Node2 → ... → LastNode → Node2 (new head)]



C Code:

```

void begin_delete() {
    struct node *ptr;
    if (head == NULL) {
        printf("\nUNDERFLOW");
    } else if (head->next == head) {
        free(head);
        head = NULL;
        printf("\nNode deleted\n");
    } else {
        ptr = head;
        while (ptr->next != head)

```

```

    ptr = ptr->next;
    ptr->next = head->next;
    free(head);
    head = ptr->next;
    printf("\nNode deleted from beginning\n");
}
}

```

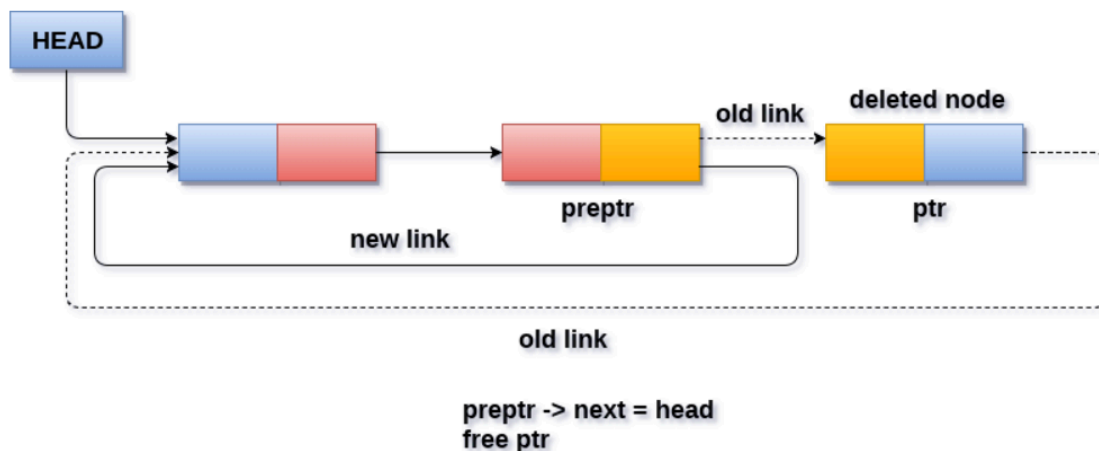
Time Complexity: $O(n)$

Explanation: Although the deletion itself is $O(1)$, we must traverse the list to find the last node to update its pointer. This traversal takes $O(n)$ time in the worst case.

Deletion at End

Deleting a node from the end of a Circular Singly Linked List involves removing the last node. If the list is empty, an underflow is reported. If there is only one node, it is freed and head is set to NULL. Otherwise, the list is traversed to find the second last node. This node's next is then set to head, and the last node is freed.

Diagram: [... → Node(n-1) → Node(n) to be deleted → Head]



C Code:

```

void last_delete() {
    struct node *ptr, *preptr;
    if (head == NULL) {

```

```

        printf("\nUNDERFLOW");
    } else if (head->next == head) {
        free(head);
        head = NULL;
        printf("\nNode deleted\n");
    } else {
        ptr = head;
        while (ptr->next != head) {
            preptr = ptr;
            ptr = ptr->next;
        }
        preptr->next = head;
        free(ptr);
        printf("\nNode deleted from end\n");
    }
}

```

Time Complexity: $O(n)$

Explanation: To delete the last node, the list must be traversed to find the second last node. This traversal requires $O(n)$ time.

Deletion at Specific Position

To delete a node at a specific position in a Circular Singly Linked List, traverse the list up to the (position - 1)th node. Adjust its next pointer to skip over the node to be deleted, then free that node. Special care is taken if the position is 1 (head node deletion).

[... $\rightarrow$ Node(i-1) $\rightarrow$ Node(i) to be deleted $\rightarrow$ Node(i+1) $\rightarrow$...]

C Code:

```

void delete_position(int pos) {
    struct node *ptr, *temp;
    int count = 1;
    if (head == NULL) {
        printf("\nUNDERFLOW");
        return;
    }
    if (pos == 1) {
        begin_delete();
        return;
    }
}

```

```

ptr = head;
while (count < pos - 1 && ptr->next != head) {
    ptr = ptr->next;
    count++;
}
if (ptr->next == head || count != pos - 1) {
    printf("Invalid position");
    return;
}
temp = ptr->next;
ptr->next = temp->next;
free(temp);
printf("Node deleted from position %d", pos);
}

```

Time Complexity: $O(n)$

Explanation: The list must be traversed to reach the (position - 1)th node, which takes $O(n)$ time. The deletion itself is $O(1)$, but traversal dominates.

Display Operation

The display operation traverses a Circular Singly Linked List and prints each node's data. Since the list has no NULL terminator, traversal continues until the starting node is reached again. This requires checking the condition that the current node has not looped back to head.

[Head $\rightarrow$ Node2 $\rightarrow$ Node3 $\rightarrow$... $\rightarrow$ LastNode $\rightarrow$ Head] (print in this order)

C Code:

```

void display() {
    struct node *ptr;
    if (head == NULL) {
        printf("\nList is empty");
        return;
    }
    ptr = head;
    printf("Circular Linked List: ");
    do {
        printf("%d ", ptr->data);
        ptr = ptr->next;
    } while (ptr != head);
}

```

Time Complexity: $O(n)$

Explanation: Each node is visited exactly once to display its data. Since there are n nodes, the time complexity is $O(n)$.

Search Operation

The search operation looks for a specific value in the Circular Singly Linked List. Traversal is done node-by-node until the value is found or the list loops back to the head. If found, the position is returned; otherwise, a message is displayed indicating the value is not present.

[Head $\rightarrow$... $\rightarrow$ TargetNode (value found) $\rightarrow$... $\rightarrow$ Head]

C Code:

```
void search(int item) {
    struct node *ptr = head;
    int pos = 1;
    if (head == NULL) {
        printf("\nList is empty");
        return;
    }
    do {
        if (ptr->data == item) {
            printf("Item %d found at position %d", item, pos);
            return;
        }
        ptr = ptr->next;
        pos++;
    } while (ptr != head);
    printf("Item %d not found in the list", item);
}
```

Time Complexity: $O(n)$

Explanation: In the worst case, the entire list must be traversed if the item is not present or is found at the last node. Thus, the time complexity is $O(n)$.

Polynomial Examples using Single Linked List

Evaluate the given polynomial Expression for the given x value:

Given a polynomial like:

$$P(x) = 4x^3 + 3x^2 + 2x + 1$$

And a value $x = 2$, evaluate the expression.

Input:

- A polynomial represented as a list of terms (each with coefficient and exponent).
- A value of x to substitute.

Algorithm

1. Start
2. Initialize result = 0
3. For each term in the polynomial:
 - a. term_value = coefficient $\times$ (x ^ exponent)
 - b. result = result + term_value
4. End For
5. Return result
6. Stop

Program using Single Linked List:

```
//Program representing polynomial term of a polynomial
//Program also calculates the value of the given polynomial expression
#include<stdio.h>
#include<stdlib.h>
#include<math.h>

//structure of the polynomial
struct node{
    float coef;
    unsigned int expo;
    struct node *next;
} *poly, *temp, *head;

//create a node of type poly
struct poly* createPoly(int n){
    int i, e;
```

```

float c;
for(i=0; i<n; i++){
    printf("\nEnter the term coefficient of order %d:\t", i);
    scanf("%f", &c);
    printf("\nEnter the term exponent of order %d:\t", i);
    scanf("%lu", &e);
    if(c == 0 || e<0){
        printf("\nYou have entered either coeff as zero\n or exponent less than
zero");
        continue;
    }

    poly = (struct node*) malloc(sizeof(struct node));
    poly->coef = c;
    poly->expo = e;
    poly->next = NULL;
    if(i==0){
        head = temp = poly;
    }
    else{
        temp->next = poly;
        temp = poly;
    }
}
}

void display(){
    temp = head;
    printf("\n\nP(x) = ");
    while(temp!=NULL){
        printf("%.2f * x^%d + ",temp->coef, temp->expo);
        temp = temp->next;
    }
}
}

```

//calculate the polynomial value

```

void polyValue(){
    float pv = 0, x;
    temp = head;
    printf("\n\n Enter the x value:\t");
    scanf("%f", &x);
    while(temp!=NULL){
        pv += temp->coef * pow(x, temp->expo);
    }
}

```

```

        temp = temp->next;
    }

    display();
    printf("\n\nThe value of the polynimial for the given %.2f as x value:\t%.2f \n\n",x,
pv);
}

int main(){
    int n;
    float polyVal;
    printf("\nEnter the number terms in the polynomial: \t");
    scanf("%d", &n);
    createPoly(n);
    polyValue();
}

```

```

Enter the number terms in the polynomial:      4
Enter the term coefficient of order 0:      4
Enter the term exponent of order 0:      3
Enter the term coefficient of order 1:      3
Enter the term exponent of order 1:      2
Enter the term coefficient of order 2:      2
Enter the term exponent of order 2:      1
Enter the term coefficient of order 3:      1
Enter the term exponent of order 3:      0

Enter the x value:      2

P(x) = 4.00 * x^3 + 3.00 * x^2 + 2.00 * x^1 + 1.00 * x^0 +
The value of the polynimial for the given 2.00 as x value:      49.00

-----
Process exited after 86.56 seconds with return value 0
Press any key to continue . . .

```

Derivative of the given Polynomial

Differentiation Rules Recap: For each term:

$$f(x) = a \cdot x^n \Rightarrow f'(x) = a \cdot n \cdot x^{n-1}$$

to evaluate the **derivative of the polynomial**

$$f(x) = 2x^3 + 4x^2 + 5x + 2$$

at $x = 2$.

Step-by-step Recap

- $\frac{d}{dx}(2x^3) = 6x^2$
- $\frac{d}{dx}(4x^2) = 8x$
- $\frac{d}{dx}(5x) = 5$
- $\frac{d}{dx}(2) = 0$

So,

$$f'(x) = 6x^2 + 8x + 5$$

2. Substitute $x = 2$:

$$f'(2) = 6(2)^2 + 8(2) + 5 = 6(\downarrow) + 16 + 5 = 24 + 16 + 5 = \boxed{45}$$

That is, multiply the coefficient by the exponent, and subtract 1 from the exponent.

Algorithm:

1. Traverse the original list.
2. For each node:
 - If exponent is not 0:
 - Create a new node with:
 - $\text{new coefficient} = \text{coef} \cdot \text{expo}$
 - $\text{new exponent} = \text{expo} - 1$
 - If exponent is 0 (constant term), skip it (derivative = 0).
3. Store the derivative in a new list.

Pseudocode:

Function DifferentiatePolynomial(poly):

Set current = poly

While current is not NULL:

 If current.expo == 0:

 // Derivative of a constant is 0

 Set current.coef = 0

 Set current.expo = 0

 Else:

 // Power rule: $d/dx (a \cdot x^n) = a \cdot n \cdot x^{(n-1)}$

 current.coef = current.coef * current.expo

 current.expo = current.expo - 1

 Move current to current.next

Return poly

//Program implementing Derivative/differentiate of polynomials

```
#include <stdio.h>
#include <stdlib.h>
#include <math.h>

// Define the node structure
struct node {
    float coef;          // Coefficient
    unsigned int expo;    // Exponent
    struct node* next;
} *head, *newnode, *temp;

static int ct=0;

// Function to create a new node
void createNewNode(float coef, unsigned int expo) {
    newnode = (struct node*)malloc(sizeof(struct node));
    newnode->coef = coef;
    newnode->expo = expo;
    newnode->next = NULL;
    if (newnode->expo!=0)
        ct++;
    if (head == NULL) {
        head = temp = newnode;
    }
    else {
        temp->next = newnode;
        temp = newnode;
    }
}

// Display polynomial
void displayPoly(int n) {
    temp = head;
    int i=0;
    while (temp != NULL && i<n) {
```

```

    printf("%.2f*x^%lu + ", temp->coef, temp->expo);
    temp = temp->next;
    i++;
}
printf("NULL\n");
}

```

```

void differentiate() {
    temp = head;
    while (temp != NULL) {
        if (temp->expo != 0) {
            temp->coef = temp->coef * temp->expo;
            temp->expo = temp->expo - 1;
        }
        temp = temp->next;
    }
}

```

```

int evaluate(int x) {
    int result = 0;
    temp = head;
    int i = 0;
    while (temp != NULL && i < ct) {
        result += temp->coef * pow(x, temp->expo);
        temp = temp->next;
        i++;
    }
    return result;
}

```

```

int main() {
    struct node* diffPoly = NULL;
    int i, n;
    float coef;

```

```

    unsigned int expo;
    printf("\nEnter the number of terms in polynomial:\n");
    scanf("%d", &n);
    for (i = 0 ; i<n ; i++){
        printf("Term %d - Coefficient and Exponent: ", i + 1);
        scanf("%f %lu", &coef, &expo);
        createNewNode(coef, expo);
    }
    printf("Original Polynomial:\n");
    displayPoly(n);
    printf("\nThe constant with no x value becomes zero\n");
    printf("\nPolynomial considered for differentiation is: \n");
    displayPoly(ct);
    differentiate();
    printf("\nAfter Differentiation:\n");
    displayPoly(ct);

    int x = 2; //assume or read 'x' value
    int value = evaluate(x);
    printf("\nValue of derivative at x = %d: %d\n", x, value);
    return 0;
}

```

```

Enter the number of terms in polynomial:
3
Term 1 - Coefficient and Exponent:  6 2
Term 2 - Coefficient and Exponent:  8 1
Term 3 - Coefficient and Exponent:  5 0
Original Polynomial:
6.00*x^2 + 8.00*x^1 + 5.00*x^0 + NULL

The constant with no x value becomes zero

Polynomial considered for differentiation is:
6.00*x^2 + 8.00*x^1 + NULL

After Differentiation:
12.00*x^1 + 8.00*x^0 + NULL

Value of derivative at x = 2: 32

-----
Process exited after 88.65 seconds with return value 0
Press any key to continue . . .

```

Addition of Two Polynomials

Add two polynomials represented as linked lists, where each node contains a **coefficient** and an **exponent**, and the list should be in **descending order of exponents**.

Input:

- poly1Head: Head pointer of first polynomial
- poly2Head: Head pointer of second polynomial

Output:

- resultPoly: Head pointer of resultant polynomial (sum)

Pseudocode:

Function AddPolynomials(poly1, poly2):

Initialize resultHead = NULL

Initialize resultTail = NULL

While poly1 is not NULL and poly2 is not NULL:

 If poly1.expo == poly2.expo:

 sumCoef = poly1.coef + poly2.coef

 If sumCoef ≠ 0:

```

        Create newNode with (sumCoef, poly1.expo)
        Append newNode to result
        Move poly1 to poly1.next
        Move poly2 to poly2.next

    Else If poly1.expo > poly2.expo:
        Create newNode with (poly1.coef, poly1.expo)
        Append newNode to result
        Move poly1 to poly1.next

    Else: // poly2.expo > poly1.expo
        Create newNode with (poly2.coef, poly2.expo)
        Append newNode to result
        Move poly2 to poly2.next
End While

// Append remaining nodes
While poly1 is not NULL:
    Create newNode with (poly1.coef, poly1.expo)
    Append newNode to result
    Move poly1 to poly1.next

While poly2 is not NULL:
    Create newNode with (poly2.coef, poly2.expo)
    Append newNode to result
    Move poly2 to poly2.next
Return resultHead

```

Program implementing the addition of any Two Polynomials:

```

#include <stdio.h>
#include <stdlib.h>

struct node {
    float coef;
    unsigned int expo;
    struct node* next;
}*poly1Head, *poly2Head, *resultPoly;

// Function to create a new node and insert
struct node* createNode(struct node **poly, int coef, int expo) {
    struct node *newnode = (struct node*)malloc(sizeof(struct node));
    newnode->coef = coef;
    newnode->expo = expo;
    newnode->next = NULL;
}

```

```

if (*poly == NULL) {
    *poly = newnode;
} else {
    struct node *temp = *poly;
    while (temp->next != NULL)
        temp = temp->next;
    temp->next = newnode;
}
return *poly;
}

void displayPoly(struct node* poly) {
    while (poly != NULL) {
        printf("%.2f*x^%lu + ", poly->coef, poly->expo);
        poly = poly->next;
    }
    printf("null\n");
}

struct node* addPolynomials(struct node* poly1Head, struct node* poly2Head) {
    struct node* result = NULL;

    while (poly1Head != NULL && poly2Head != NULL) {
        if (poly1Head->expo == poly2Head->expo) {
            int sum = poly1Head->coef + poly2Head->coef;
            if (sum != 0)
                createNode(&result, sum, poly1Head->expo);
            poly1Head = poly1Head->next;
            poly2Head = poly2Head->next;
        }
        else if (poly1Head->expo > poly2Head->expo) {
            createNode(&result, poly1Head->coef, poly1Head->expo);
            poly1Head = poly1Head->next;
        }
        else {
            createNode(&result, poly2Head->coef, poly2Head->expo);
            poly2Head = poly2Head->next;
        }
    }

    // Remaining terms
    while (poly1Head != NULL) {
        createNode(&result, poly1Head->coef, poly1Head->expo);
        poly1Head = poly1Head->next;
    }
    while (poly2Head != NULL) {
        createNode(&result, poly2Head->coef, poly2Head->expo);
        poly2Head = poly2Head->next;
    }
    return result;
}

```

```

int main() {
    int n, i;
    float coef;
    unsigned int expo;
    printf("Enter number of terms in first polynomial: ");
    scanf("%d", &n);
    for (i = 0; i < n; i++) {
        printf("Term %d - Coefficient and Exponent: ", i + 1);
        scanf("%f %lu", &coef, &expo);
        poly1Head = createNode(&poly1Head, coef, expo);
    }
    printf("Enter number of terms in second polynomial: ");
    scanf("%d", &n);
    for (i = 0; i < n; i++) {
        printf("Term %d - Coefficient and Exponent: ", i + 1);
        scanf("%f %lu", &coef, &expo);
        poly2Head = createNode(&poly2Head, coef, expo);
    }
    printf("\nFirst Polynomial: ");
    displayPoly(poly1Head);
    printf("\n\nSecond Polynomial: ");
    displayPoly(poly2Head);
    resultPoly = addPolynomials(poly1Head, poly2Head);
    printf("Sum of Polynomials: ");
    displayPoly(resultPoly);
    return 0;
}

```

```

Enter number of terms in first polynomial: 3
Term 1 - Coefficient and Exponent: 3 2
Term 2 - Coefficient and Exponent: 2 1
Term 3 - Coefficient and Exponent: 1 0
Enter number of terms in second polynomial: 2
Term 1 - Coefficient and Exponent: 4 3
Term 2 - Coefficient and Exponent: 5 2

First Polynomial: 3.00x^2 + 2.00x^1 + 1.00x^0 + null

Second Polynomial: 4.00x^3 + 5.00x^2 + null
Sum of Polynomials: 4.00x^3 + 8.00x^2 + 2.00x^1 + 1.00x^0 + null

-----
Process exited after 28.74 seconds with return value 0
Press any key to continue . . .

```

Enter number of terms in first polynomial: 3

Term 1 - Coefficient and Exponent: 3 2

Term 2 - Coefficient and Exponent: 2 1

Term 3 - Coefficient and Exponent: 1 0

Enter number of terms in second polynomial: 3

Term 1 - Coefficient and Exponent: 4 2

Term 2 - Coefficient and Exponent: 5 1

Term 3 - Coefficient and Exponent: 6 0

First Polynomial: $3.00x^2 + 2.00x^1 + 1.00x^0 + \text{null}$

Second Polynomial: $4.00x^2 + 5.00x^1 + 6.00x^0 + \text{null}$

Sum of Polynomials: $7.00x^2 + 7.00x^1 + 7.00x^0 + \text{null}$

Process exited after 65.05 seconds with return value 0

Press any key to continue . . .

Radix Sort

Radix Sort is a non-comparative sorting algorithm that sorts numbers digit by digit, starting from the least significant digit (LSD) to the most significant digit (MSD). It is often used to sort integers or strings of fixed length. Radix Sort uses a stable sorting algorithm (like Counting Sort) to sort digits in each position, one digit at a time.

How it work?

Let's say we are sorting decimal numbers (base 10), and each number has d digits.

- Start with the least significant digit.
- Use a stable sort (e.g., Counting Sort) to sort the numbers based on that digit.
- Move to the next significant digit and repeat.
- After d passes, the numbers are sorted.

Let's explain with an example.

Example For example consider the following list. Observe that we have prepended 0s as necessary to equalize the lengths.

170,045,075,090,802,024,002,066

First we identify each right-most (least significant) digit:

170, 045, 075, 090, 802, 024, 002, 066

We sort the list according to that digit, preserving order within that group:

170, 090, 802, 002, 024, 045, 075, 066

Then we identify the next digit, moving leftwards:

170, 090, 802, 002, 024, 045, 075, 066

We sort the list according to that digit, preserving order within that group:

802, 002, 024, 045, 066, 170, 075, 090

Then we identify the next digit, moving leftwards:

802, 002, 024, 045, 066, 170, 075, 090

We sort the list according to that digit, preserving order within that group:

002, 024, 045, 066, 075, 090, 170, 802,

Why Counting Sort?

- In Radix Sort, we sort the list digit by digit (from least to most significant digit).
- At each step, we only focus on one digit of each number — the rest of the digits "just go along for the ride".

- Each digit is a number between 0 and $b - 1$, where b is the base (usually 10 for decimal numbers).

So, at each step, we are sorting n items (where n is the number of elements) using only one digit — and that digit lies in a small fixed range: 0 to $b - 1$.

Because of this, Counting Sort is a perfect fit:

- It's stable.
- It works well when the range of values is small (which is true for digits).

Counting Sort:

Counting sort assumes that each of the n input elements is an integer in the range 0 to k , for some integer k . Counting sort determines, for each input element x , the number of elements less than x . It uses this information to place element x directly into its position in the output array. For example, if 17 elements are less than x , then x belongs in output position 18. We must modify this scheme slightly to handle the situation in which several elements have the same value, since we do not want to put them all in the same position.

Assumption:

Each element in the input array $A[1 \dots n]$ is an integer in the range $[0 \dots k]$, where k is a known upper bound.

We use two auxiliary arrays:

- $B[1 \dots n]$: To store the final sorted output.
- $C[0 \dots k]$: To store frequencies and prefix sums for placement.

How it Works (Step-by-Step)

1. Count the Occurrences

For each value x in A , increment $C[x]$. This gives the count of how many times each value appears.

2. Cumulative Sum (Prefix Sum)

For each index i in C , compute: $C[i] = C[i] + C[i-1]$

Now, $C[i]$ tells us how many elements are less than or equal to i . So, if $C[5] = 17$, then all elements ≤ 5 occupy positions 1 to 17.

3. Place Elements in Output Array (B)

Loop through A from end to start (for stability):

- Let $x = A[j]$
- Place x at position $C[x]$ in B
- Decrement $C[x]$ by 1

4. Result

Now B contains all the elements in sorted order.

COUNTING-SORT(A, B, k)

1. let $C[0..k]$ be a new array
2. for $i = 0$ to k
 $C[i] = 0$
3. for $j = 1$ to $A.length$
 $C[A[j]] = C[A[j]] + 1$
4. for $i = 1$ to k
 $C[i] = C[i] + C[i - 1]$
5. for $j = A.length$ downto 1
 $B[C[A[j]]] = A[j]$
 $C[A[j]] = C[A[j]] - 1$

For loop at (2) takes time $\theta(k)$, the for loop at (3) takes time $\theta(n)$, the for loop at (4) takes time $\theta(k)$, and for loop at (5) takes $\theta(n)$ times. Thus, the overall time is $\theta(k+n)$. We usually use counting sort when we have $k = O(n)$, in which case the running time is $\theta(n)$.

If this is the case then each counting sort in Radix sort step takes $\theta(n+(b-1))$ times, and we are doing d of those so the time complexity is $\theta(d(n+(b-1))) = \theta(dn)$.